

TRABAJO INTEGRADOR

Modelado e implementación de una aplicación con Diseño Orientada a Objetos

Datos del trabajo

Modalidad: Grupal — 4 a 5 integrantes

Tiempo de desarrollo: 3 a 4 semanas

Lenguaje: Java o Python (a elección del grupo)

Entregables: Diagrama UML + Código fuente + Informe técnico

1. Descripción general

El trabajo final integrador tiene como objetivo que el grupo diseñe e implemente una aplicación orientada a objetos de mediana complejidad, aplicando de forma coherente y justificada los principios y mecanismos del paradigma trabajados durante la cursada.

La aplicación debe resolver un problema real, estar correctamente modelada mediante un diagrama de clases UML y ser implementada en Java o Python respetando las buenas prácticas del paradigma.

2. Requisitos del sistema a desarrollar

El grupo debe proponer y desarrollar un sistema de gestión propio. El sistema elegido debe incluir los siguientes requisitos estructurales:

a. Jerarquía de herencia

El modelo debe incluir al menos una clase abstracta con dos o más subclases concretas que especialicen su comportamiento. La herencia debe estar justificada por una relación "es un" real dentro del dominio.

b. Encapsulamiento

Todos los atributos deben ser privados o protegidos. El acceso debe realizarse mediante métodos públicos. Ningún atributo debe ser accesible directamente desde fuera de la clase.

c. Polimorfismo

Al menos un método debe ser sobrescrito en las subclases con comportamiento distinto. El sistema debe invocar ese método de forma polimórfica, es decir, sin conocer el tipo concreto del objeto en el momento de la llamada.

d. Composición o agregación

Incluir relaciones mediante composición o agregación, con multiplicidades definidas y justificadas.

e. Colecciones

El sistema debe gestionar al menos una colección de objetos (lista, conjunto o mapa), con operaciones de alta, búsqueda y recorrido.

f. Clase de reporte o comprobante

El sistema debe incluir al menos una clase cuya responsabilidad sea consolidar y presentar información de otros objetos, sin duplicar datos sino delegando a los objetos que los poseen.

g. Constructores

Las clases deben inicializarse correctamente mediante constructores. Al menos una clase debe tener más de un constructor (sobrecarga).

⚠ Validación de la propuesta

El grupo debe presentar la propuesta al docente para su validación antes de comenzar el desarrollo. No se aceptarán trabajos sobre sistemas no validados.

4. Entregables

El grupo debe entregar los siguientes tres componentes. Todos son obligatorios; la ausencia de cualquiera implica la no aprobación del trabajo.

4.1 Diagrama de clases UML

- Diagrama completo con todas las clases del sistema.
- **Debe incluir:** nombre de la clase, atributos (con tipo y visibilidad), métodos (con parámetros y tipo de retorno), relaciones (herencia, composición, agregación, asociación) con multiplicidades correctamente indicadas.
- Las clases abstractas deben estar marcadas como «abstract».
- **Formato:** imagen (PNG o PDF) o herramienta de diagramación (draw.io, Lucidchart, StarUML u otras).

4.2 Código fuente

- Implementación completa y funcional en Java o Python.
- El código debe ejecutarse sin errores.
- **Debe incluir un programa principal (main o equivalente)** que implemente una interfaz gráfica de usuario (GUI) desarrollada con Java Swing o Python Tkinter según el lenguaje elegido. El funcionamiento completo del sistema debe poder demostrarse exclusivamente a través de la GUI, sin necesidad de interactuar con la consola.
- **Se valorará:** uso de comentarios explicativos, nombres descriptivos, código limpio y sin redundancias.

4.3 Informe técnico

El informe debe incluir las siguientes secciones:

1. **Carátula:** nombre del sistema, integrantes del grupo, legajos, fecha de entrega.
2. **Descripción del sistema:** qué problema resuelve, quiénes son los usuarios y qué funcionalidades ofrece.
3. **Decisiones de diseño:** justificación de la jerarquía de clases, las relaciones elegidas y las multiplicidades.
4. **Diagrama UML** (puede incluirse aquí o como anexo).
5. **Conclusiones:** dificultades encontradas, decisiones que cambiarían y aprendizajes del grupo.

5. Criterios de evaluación

Criterio	Peso
Corrección y completitud del diagrama UML	20%
Aplicación correcta de los principios POO en el código	35%
Funcionamiento del programa (compila, ejecuta, produce resultados correctos)	20%
Calidad del informe técnico (claridad, justificación de decisiones)	15%
Organización del código (nombres, estructura, comentarios)	10%

6. Condiciones de entrega y defensa

- **Fecha límite de entrega y Defensa:** 22 y 29 de Julio de 2026
- **Formato de entrega:** repositorio GitHub con el código fuente(revisar permisos para que el docente tenga acceso), el diagrama UML y el informe en PDF. El link debe enviarse por el campus virtual antes de la fecha de presentación.
- **Defensa oral:** cada grupo realizará una presentación de 15 -20 minutos donde explicará las decisiones de diseño y responderá preguntas del docente. **Todos los integrantes deben poder explicar cualquier parte del trabajo.**
- **Condición de aprobación:** el trabajo se aprueba si obtiene un puntaje igual o superior al 60% del total.

7. Recomendaciones

💡 Consejos para un buen trabajo

- Comiencen por el diagrama UML antes de escribir una sola línea de código. Un buen diseño hace que la implementación sea mucho más sencilla.
- Dividan el trabajo por clases o módulos, no por capas. Es más fácil que cada integrante sea responsable de un conjunto de clases relacionadas.
- Usen GitHub desde el primer día para versionar el trabajo y evitar pérdida de código.
- Consulten al docente ante cualquier duda de diseño antes de implementar: es mucho más costoso corregir el código que corregir el diagrama.
- Validen la propuesta antes de iniciar: cuanto antes tengan el visto bueno del docente, más tiempo tienen para desarrollar.